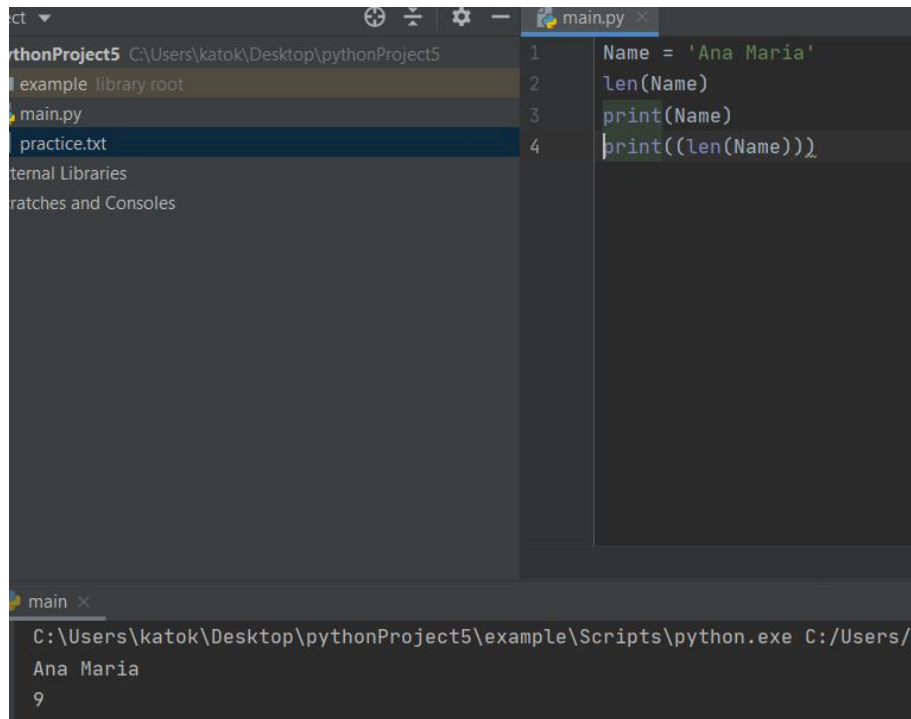


Lecture 1:

Exercise 1: WAP to input user's first name & print its length



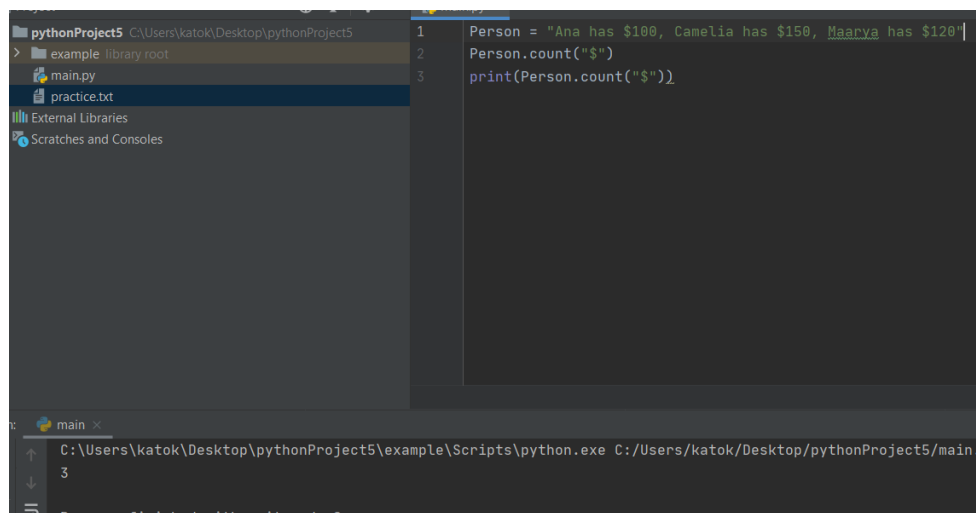
```
pythonProject5 C:\Users\katok\Desktop\pythonProject5
example library root
main.py
practice.txt
External Libraries
Scratches and Consoles

1 Name = 'Ana Maria'
2 len(Name)
3 print(Name)
4 print((len(Name)))

main x
C:\Users\katok\Desktop\pythonProject5\example\Scripts\python.exe C:/Users/k
Ana Maria
9
```

I've create a variable called "name" and stored the text "Ana Maria". Since it is in "", the text is a string. I used the function **len** to calculate the lengh of the string. The Output = 9.

Exercise 2: WAP to find the occurrence of '\$' in a String.



```
pythonProject5 C:\Users\katok\Desktop\pythonProject5
example library root
main.py
practice.txt
External Libraries
Scratches and Consoles

1 Person = "Ana has $100, Camelia has $150, Maarya has $120"
2 Person.count("$")
3 print(Person.count("$"))

main x
C:\Users\katok\Desktop\pythonProject5\example\Scripts\python.exe C:/Users/katok/Desktop/pythonProject5/main.
3
Process finished with exit code 0
```

I've created a variable called **person** and stored a string inside it. The **count()** method, counts how many time something appears in a string. We want to know how many times, the symbol \$ appears in the string, so the Output = 3.

Exercise 3: WAP to check if a number entered by the user is odd or even.

```
number = 3

if number % 2 == 0:
    print("It is even")
else:
    print("It is odd")
```

I've created a variable called number and stored number 3 in it. Second line is the condition. The % gives the reminder after division. So in this code the Output = It is odd, while in the example bellow, the Output = It is even.

```
number = 4

if number % 2 == 0:
    print("It is even")
else:
    print("It is odd")
```

Exercise 4: WAP to find the greatest of 3 numbers entered by the user.

```
number1 = 5
number2 = 15
number3 = 8

if number1 > number2 and number1 > number3:
    print("Number 1 is greatest")
elif number2 > number1 and number2 > number3:
    print("Number 2 is the greatest!")
else:
    print("Number 3 is greatest!!")
```

I've created 3 variables, the **if** statement checks whether **number1** is the largest number. Because of the **and** operator, both conditions must be true. In case the previous condition

was false, **elif** is checking if it's condition is true. And **else** is executed only if none of the previous conditions was true. But in this case, the second condition was true, so the Output = Number 2 is the greatest.

Exercise 5: WAP to check if a number is a multiple of 7 or not.

```
number = 51

if number % 7 == 0:
    print("It is multiple of 7!")
else:
    print("It is not multiple of 7!")
```

I've created a variable called **number** and stored the value **51**. The second line checks if number 51 is divided by 7. Because it is not divided by 7, the Output = It is not multiple of 7.

Exercise 6: WAP to ask the user to enter names of their 3 favorite movies & store them in a list.

```
movie1 = input("Enter your first favourite movie.")
movie2 = input("Enter your second favourite movie.")
movie3 = input("Enter your third favourite movie.")

my_movies = [movie1, movie2, movie3]
print(my_movies)
```

I've asked the user to type something, using the **input** command. Whatever the user is typing, it will be stored inside movie1 and so on for the other two. **My_movies** creates a list with the movies that the user has input.

Exercise 7: WAP to check if a list contains a palindrome of elements. (Hint: use copy() method)

```

numbers = [2, 5, 1, 7]

copy_numbers = numbers.copy()
copy_numbers.reverse()

if numbers == copy_numbers:
    print("Yes!")
else:
    print("No!")

```

I've create a list called numbers, using the **copy()** method, I have copied the list. I've used **reverse()** in order to reverse the list and to check if it contains a palindrome of elements and the Output = No!

Exercise 8.1: WAP to count the number of students with the “A” grade in the following tuple. [”C”, “D”, “A”, “A”, “B”, “B”, “A”]

```

grades = ("C", "D", "A", "A", "B", "B", "A")
grades.count("A")

print(grades.count("A"))

```

I've created a tuple. So we want to know how many times the letter "A" appears in the tuple. For this I have used the method **.count** . So the Output = 3.

Exercise 8.2: Store the above values in a list & sort them from “A” to “D”.

```

grades = ["C", "D", "A", "A", "B", "B", "A"]

grades.sort()
print(grades)

```

By using the method **.sort()** I've put the list in order, so the Output = ['A', 'A', 'A', 'B', 'B', 'C', 'D']

Exercise 9: Store following word meanings in a python dictionary : table : “a piece of furniture”, “list of facts & figures” cat : “a small animal.”

```
my_dict = {  
    "Table": ["A piece of furniture", "List of facts & figures"],  
    "Cat": "A small animal."  
}  
  
print(my_dict)
```

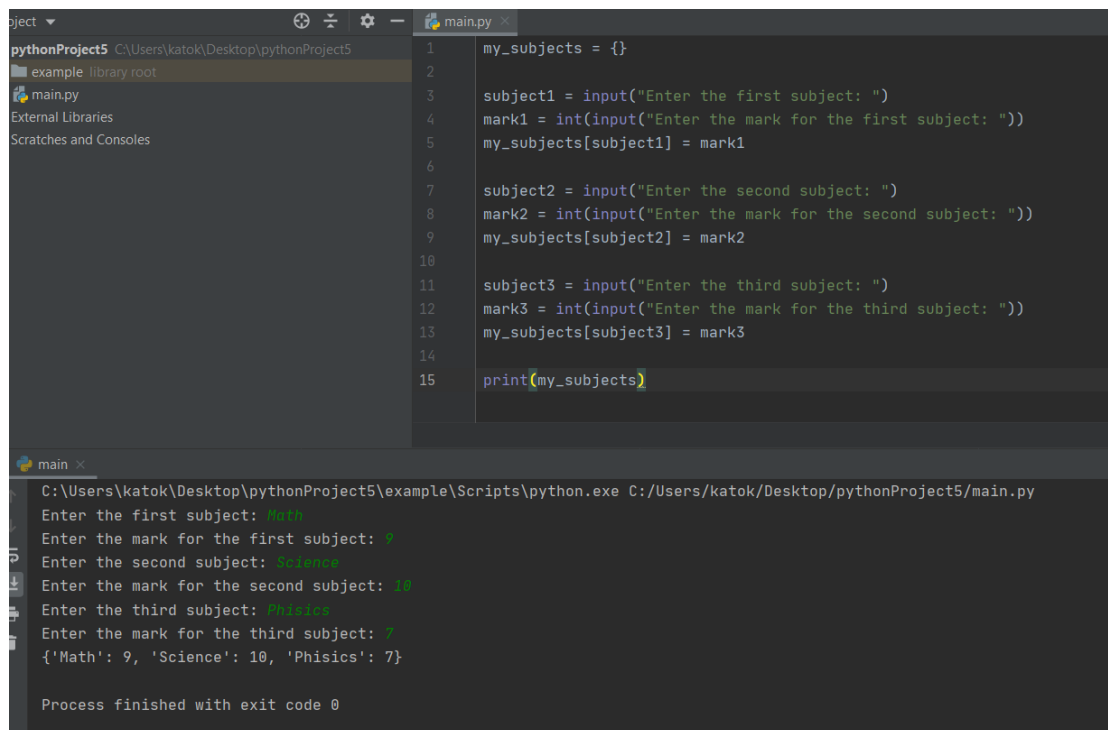
I've created a dictionary that contains key-value pairs. At the key **Table** I've created a list containing two meanings. And after that I have printed the Output.

Exercise 10: You are given a list of subjects for students. Assume one classroom is required for 1 subject. How many classrooms are needed by all students. "python", "java", "C++", "python", "javascript", "java", "python", "java", "C++", "C"

```
subject = ["python", "java", "C++", "python", "javascript", "java", "python", "java", "C++", "C"]  
set(subject)  
subject = set(subject)  
print(len(subject))  
|
```

I've created a list called subject. Second line converts the list into a set, so it can remove the duplicates, last line prints the length of the set and the Output = 5.

Exercise 11: WAP to enter marks of 3 subjects from the user and store them in a dictionary. Start with an empty dictionary & add one by one. Use subject name as key & marks as value.



```
1 my_subjects = {}
2
3 subject1 = input("Enter the first subject: ")
4 mark1 = int(input("Enter the mark for the first subject: "))
5 my_subjects[subject1] = mark1
6
7 subject2 = input("Enter the second subject: ")
8 mark2 = int(input("Enter the mark for the second subject: "))
9 my_subjects[subject2] = mark2
10
11 subject3 = input("Enter the third subject: ")
12 mark3 = int(input("Enter the mark for the third subject: "))
13 my_subjects[subject3] = mark3
14
15 print(my_subjects)
```

main

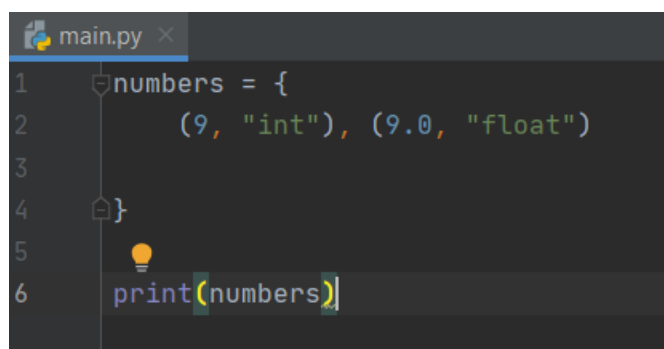
C:\Users\katok\Desktop\pythonProject5\example\Scripts\python.exe C:/Users/katok/Desktop/pythonProject5/main.py

Enter the first subject: Math
Enter the mark for the first subject: 9
Enter the second subject: Science
Enter the mark for the second subject: 10
Enter the third subject: Physics
Enter the mark for the third subject: 7
{'Math': 9, 'Science': 10, 'Physics': 7}

Process finished with exit code 0

This program creates an empty dictionary to store three subjects and their marks. It asks the user to enter each subject name and its mark, then saves them in the dictionary. Finally, it prints the dictionary showing all the subjects with their marks.

Exercise 12: Figure out a way to store 9 & 9.0 as separate values in the set. (You can take help of built-in data types)



```
1 numbers = {
2     (9, "int"), (9.0, "float")
3 }
4
5
6 print(numbers)
```

I've created a **set** called `numbers` that contains two tuples. Each tuple stores a number and a string describing its data type. The first tuple contains the integer 9 and the string "int", while the second tuple contains the decimal number 9.0 and the string "float". Finally, the `print()` function displays the contents of the set on the screen.

Exercise 13: Print the elements of the following list using a loop: [1, 4, 9, 16, 25, 36, 49, 64, 81, 100]

```
main.py x
1 numbers = [1, 4, 9, 16, 25, 36, 49, 64, 81, 100]
2 for item in numbers:
3     print(item)
```

I've created a **list** of numbers and uses a **for loop** to go through each item in the list. During each iteration, the current number is stored in the variable `item` and displayed on the screen using the `print()` function.

Exercise 14:

```
main.py x
1 for numbers in range(100, 0, -1):
2     print(numbers)
3
```

I've created a **for loop** to print the numbers from **100 down to 1**. The `range(100, 0, -1)` function counts backwards by **1**, and `print()` displays each number on the screen.

Exercise 15: Print numbers from 1 to 100.

```
main.py x
1 for numbers in range(1, 101):
2     print(numbers)
```

I've created a loop to print the number 1 to 100.

The image shows a Python IDE interface with a file explorer on the left and a code editor on the right. The file explorer shows a project named 'pythonProject5' with a subdirectory 'example' containing a file 'main.py'. The code editor displays the following Python code:

```
1 numbers = (1, 4, 9, 16, 25, 36, 49, 64, 81, 100)
2 search = int(input("Please input a number:"))
3
4 for item in numbers:
5     if item == search:
6         print("Number exists in tuple.")
7         break
8     else:
9         print("Number not found.")
10
```

Below the code editor is a console window showing the execution of the program. The command prompt shows the path to the Python executable and the script file. The input '50' is entered, and the output 'Number not found.' is displayed. The process finished with exit code 0.

```
main x
C:\Users\katok\Desktop\pythonProject5\example\Scripts\python.exe C:/Users/katok/Desktop/pythonProject5/main.py
Please input a number:50
Number not found.
Process finished with exit code 0
```

The screenshot shows an IDE with a project named 'pythonProject5'. The file explorer on the left shows 'example' (library root), 'main.py', and 'practice.txt'. The main editor displays the following Python code in 'main.py':

```
1 def find_word():
2     file = open("practice.txt", "r")
3
4     line_no = 1
5     for line in file:
6         if "learning" in line:
7             print(line_no)
8             file.close()
9             return
10        line_no += 1
11
12    file.close()
13    print(-1)
14    find_word()
```

The console output shows the command: `C:\Users\katok\Desktop\pythonProject5\example\Scripts\python.exe C:/Users/katok/Desktop/pythonProject5/main.py` and the output: `2`.

The screenshot shows the same IDE with the file explorer showing 'example' (library root), 'main.py', and 'practice.txt'. The main editor displays the following Python code in 'main.py':

```
1 file = open("practice.txt", "r")
2 data = file.read()
3
4 if "learning" in data:
5     print("The word 'learning' exists in the file.")
6 else:
7     print("The word 'learning' does not exist in the file.")
8
9 file.close()
```

The console output shows the command: `C:\Users\katok\Desktop\pythonProject5\example\Scripts\python.exe C:/Users/katok/Desktop/pythonProject5/main.py` and the output: `The word 'learning' exists in the file.`

The screenshot shows the IDE with the file explorer showing 'main.py' and 'practice.txt'. The main editor displays the contents of 'practice.txt':

```
1 Hi everyone
2 we are learning File I/O
3 using Python.
4 I like programming in Python.
5
```

```
main.py x practice.txt x
1 file = open("practice.txt", "r")
2 data = file.read()
3 file.close()
4 new_data = data.replace("Java", "Python")
5 file = open("practice.txt", "w")
6 file.write(new_data)
7 file.close()
```

pythonProject5 C:\Users\katok\Desktop\pythonProject5

- example library root
- main.py
- practice.txt
- External Libraries

```
1 Hi everyone
2 we are learning File I/O
3 using Java.
4 I like programming in Java.
5
```

Project

- pythonProject5 C:\Users\katok\Desktop\pythonProject5
 - example library root
 - main.py
 - practice.txt
 - External Libraries
 - Scratches and Consoles

```
1 file = open("practice.txt", "w")
2
3 file.write("Hi everyone\n")
4 file.write("we are learning File I/O\n")
5 file.write("using Java.\n")
6 file.write("I like programming in Java.\n")
7
8 file.close()
9
```

The image shows a Python IDE with a project named 'pythonProject5' located at 'C:\Users\katok\Desktop\pythonProject5'. The file explorer on the left shows a folder 'example' with a subfolder 'library root' and a file 'main.py'. The main editor window displays the following Python code in 'main.py':

```
1 def print_list(my_list, index):
2     if index == len(my_list):
3         return
4
5     print(my_list[index])
6     print_list(my_list, index + 1)
7
8 numbers = [1, 4, 9, 16, 25]
9
10 print_list(numbers, 0)
```

The output console at the bottom shows the execution of the program. The command executed is 'C:\Users\katok\Desktop\pythonProject5\example\Scripts\python.exe C:/Users/katok/Desktop/p'. The output is:

```
1
4
9
16
25
```

The process finished with exit code 0.

The image shows a Python IDE interface with a dark theme. On the left, a file explorer shows a project named 'pythonProject5' located at 'C:\Users\katok\Desktop\pythonProject5'. Inside the project, there is a folder 'example' which contains a file 'main.py'. The 'main.py' file is selected and its contents are displayed in the main editor area. The code defines a recursive function 'sum(n)' that returns 0 if 'n' is 0, and 'n + sum(n - 1)' otherwise. Below the function definition, there is a call to 'print(sum(5))'. A lightbulb icon is visible next to the function definition, indicating a suggestion or hint. At the bottom of the IDE, a console window shows the command 'C:\Users\katok\Desktop\pythonProject5\example\Scripts\python.exe C:/Users...' and the output '15'.

```
pythonProject5 C:\Users\katok\Desktop\pythonProject5
├── example
│   ├── library root
│   └── main.py
├── External Libraries
└── Scratches and Consoles
```

```
1 def sum(n):
2     if n == 0:
3         return 0
4     return n + sum(n - 1)
5
6 print(sum(5))
```

```
C:\Users\katok\Desktop\pythonProject5\example\Scripts\python.exe C:/Users...
15
```

pythonProject5 C:\Users\katok\Desktop\pythonProject5

example library root

main.py

External Libraries

Scratches and Consoles

```
1 def usd_to_inr(usd):
2     inr = usd * 86
3     print(inr)
4
5     amount = int(input("Enter amount in USD: "))
6     usd_to_inr(amount)
```

main

C:\Users\katok\Desktop\pythonProject5\example\Scripts\python.exe C:/Users/katok/Desktop/pythonP

Enter amount in USD: 100

8600

pythonProject5 C:\Users\katok\Desktop\pythonProject5

example library root

main.py

External Libraries

Scratches and Consoles

```
1 def factorial(n):
2     fact = 1
3
4     for item in range(1, n + 1):
5         fact = fact * item
6
7     print(fact)
8
9     factorial(5)
```

factorial() → for item in range(1, n + 1)

main

C:\Users\katok\Desktop\pythonProject5\example\Scripts\python.exe C:/Users/katok/Desk

120

The image shows a Python IDE interface with a file explorer on the left, a code editor in the center, and a console at the bottom.

File Explorer: Shows a project named `pythonProject5` located at `C:\Users\katok\Desktop\pythonProject5`. Inside, there is an `example` folder (library root) containing the file `main.py`.

Code Editor: Displays the contents of `main.py`:

```
1 def print_list(my_list):
2     for item in my_list:
3         print(item, end=" ")
4
5     numbers = [1, 4, 9, 16, 25]
6     print_list(numbers)
```

Console: Shows the execution output:

```
main x
C:\Users\katok\Desktop\pythonProject5\example\Scripts\python.exe C:/Users/katok/De
1 4 9 16 25
Process finished with exit code 0
```

```
PythonProject5 C:\Users\katok\Desktop\pythonProject5
example library root
main.py
Internal Libraries
atches and Consoles

1 def print_len(my_list):
2     print(len(my_list))
3
4     numbers = [1, 4, 9, 16, 25]
5     print_len(numbers)
6

main ×
C:\Users\katok\Desktop\pythonProject5\example\Scripts\python.exe C:/Users/katok/
5

Process finished with exit code 0
```

```
pythonProject5 C:\Users\katok\Desktop\pythonProject5
example library root
main.py
External Libraries
Scratches and Consoles

1 number = int(input("Enter a number: "))
2
3 factorial = 1
4
5 for item in range(1, number + 1):
6     factorial = factorial * item
7
8 print(factorial)

main x
C:\Users\katok\Desktop\pythonProject5\example\Scripts\python.exe C:/Users/katok/Desktop/py
Enter a number: 6
720
```

```
pythonProject5 C:\Users\katok\Desktop\pythonProject5
example library root
main.py
External Libraries
Scratches and Consoles

1 number = int(input("Enter a number: "))
2
3 i = 1
4 sum = 0
5
6 while i <= number:
7     sum = sum + i
8     i = i + 1
9
10 print(sum)

while i <= number

main x
C:\Users\katok\Desktop\pythonProject5\example\Scripts\python.exe C:/Users/katok/Desktop/pythonProje
Enter a number: 3
6
Process finished with exit code 0
```

pythonProjects / main.py

Project ▾

- pythonProject5 C:\Users\katok\Desktop\pythonProject5
 - example library root
 - main.py
 - External Libraries
 - Scratches and Consoles

```
1 n = int(input("Input a number:"))
2 for i in range(1, 11):
3     print(n*i)
```

for i in range(1, 11)

Run: main ×

Input a number: 3

3
6
9
12
15
18
21
24
27
30

Process finished with exit code 0

```
1 for item in range(100, 0, -1):
2     print(item)
```

```
1 for item in range(1, 101):
2     print(item)
```